- Part 3 - Heuristics

Numerical Methods and Optimization

May 5, 2025 — ENSIA

Heuristics

Unlike exact algorithms, a heuristic does not guarantee optimality but provides a fast way to explore the solution space and find acceptable results.

Basic Definitions

- **Heuristic**: A constructive or improvement method that delivers an approximate solution.
- **Exact algorithm**: A method that guarantees the optimal solution (e.g., linear programming, Branch and Bound).
- **Metaheuristic**: A general adaptive framework to build powerful heuristics (e.g., simulated annealing, tabu search).

Classical Optimization Problems

- Travelling Salesman Problem (TSP)
- Knapsack Problem
- Assignment and Scheduling Problems
- Graph Coloring

1. Greedy Heuristic

Principle A greedy heuristic builds a solution step-by-step by always choosing the best immediate (local) option without considering the global consequences.

Algorithm Structure

1. Initialize an empty or partial solution.
2. At each step, choose the best available element according to a criterion.
3. Add it to the solution if it is feasible.
4. Repeat until a complete solution is built.

Example: Nearest Neighbor for TSP

- Start from a random city.
- At each step, go to the nearest unvisited city.
- Repeat until all cities are visited.
- Return to the starting city.

Pros: Fast and easy to implement.
Cons: May give poor solutions in some cases (myopic choices).

2. Local Search

Principle Start from an initial solution and iteratively move to a neighboring solution that improves the objective function.

Key Components

- **Initial solution:** Can be random or generated by a greedy algorithm.
- **Neighborhood:** The set of solutions obtained by small modifications (e.g., 2-opt for TSP: swapping two cities).
- **Evaluation:** Compare neighbors using the objective function.
- **Stopping criterion:** Stop when no better neighbor is found or after a number of iterations.

3. Descent Method (Hill Climbing)

Principle A simple form of local search where only improving moves are accepted. If no improving neighbor is found, the algorithm stops (local minimum).

Algorithm (Descent)

1. Generate an initial solution x.
2. While there exists a neighbor $x' \in N(x)$ such that $f(x') < f(x)$:
   - Set $x \leftarrow x'$ (choose the best or first improving neighbor).
3. Return x as the final solution.

Stopping Criteria

- No improvement in the neighborhood (local optimum).
- Maximum number of iterations or time reached.

4. Example: TSP with 2-opt Local Search

Problem Given a set of cities and distances between them, find a minimal-length tour that visits each city exactly once and returns to the start.

2-opt Neighborhood

- Select two non-adjacent edges.
- Remove them and reconnect the tour by reversing the order of the intermediate cities.

2-opt Algorithm (Sketch)

1. Start from a greedy tour (e.g., Nearest Neighbor).
2. For all pairs of edges $(i, i+1), (j, j+1)$, check if reversing the segment between $i+1$ and j reduces the total distance.
3. If yes, perform the swap.
4. Repeat until no further improvement is possible.

Conclusion Deterministic heuristics like greedy and local search offer simple and effective tools for obtaining good solutions quickly. However, they are often trapped in local optima, which motivates the need for more advanced metaheuristics.

Practical Activity

City Coordinates

We consider 5 cities located on a 2D plane as follows:

City	Coordinates (x, y)
A	(0, 0)
B	(2, 3)
C	(5, 4)
D	(1, 6)
E	(7, 1)

Distance Matrix

The Euclidean distances between cities are calculated and rounded to two decimals:

	A	B	C	D	E
A	0.00	3.61	6.40	6.08	7.07
B	3.61	0.00	3.16	3.16	5.10
C	6.40	3.16	0.00	4.12	3.16
D	6.08	3.16	4.12	0.00	7.21
E	7.07	5.10	3.16	7.21	0.00

Step 1: Nearest Neighbor Heuristic (starting at A)

- Start at A
- Closest to A: B (3.61)
- From B: closest unvisited = C (3.16)
- From C: closest unvisited = E (3.16)
- From E: last unvisited = D (7.21)
- Return to A from D = 6.08

Tour: $A \rightarrow B \rightarrow C \rightarrow E \rightarrow D \rightarrow A$

Total cost: $3.61 + 3.16 + 3.16 + 7.21 + 6.08 = \boxed{23.22}$

Step 2: Apply 2-opt Improvement

Try swapping segments to reduce the tour cost:

Attempt 1: Swap (C → E → D) with (C → D → E)

New tour: $A \to B \to C \to D \to E \to A$

Total cost: 3.61 + 3.16 + 4.12 + 7.21 + 7.07 = 25.17 (worse)

Attempt 2: Swap (B → C) and (E → D)

New tour: $A \to B \to E \to C \to D \to A$

Total cost: $3.61 + 5.10 + 3.16 + 4.12 + 6.08 = \boxed{22.07}$ (improved)

Conclusion

Method	Tour	Cost
Nearest Neighbor	A → B → C → E → D → A	23.22
After 2-opt	A → B → E → C → D → A	22.07

The 2-opt method improved the solution obtained from the Nearest Neighbor heuristic.

Overview

- Local search-based improvement techniques
- Strategies for intensification and diversification
- Methods:
  - Multi-start
  - Hill Climbing
  - GRASP (Greedy Randomized Adaptive Search Procedure)
- Key concept: **Exploration vs Exploitation**
- Application to permutation problems

Multi-start

Principle:

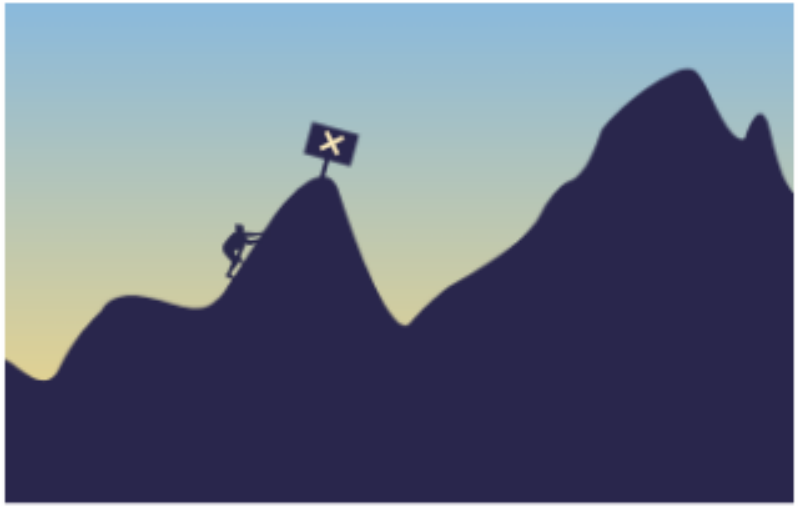
- Run a local search multiple times from different randomly generated initial solutions.
- Keep the best solution found overall.

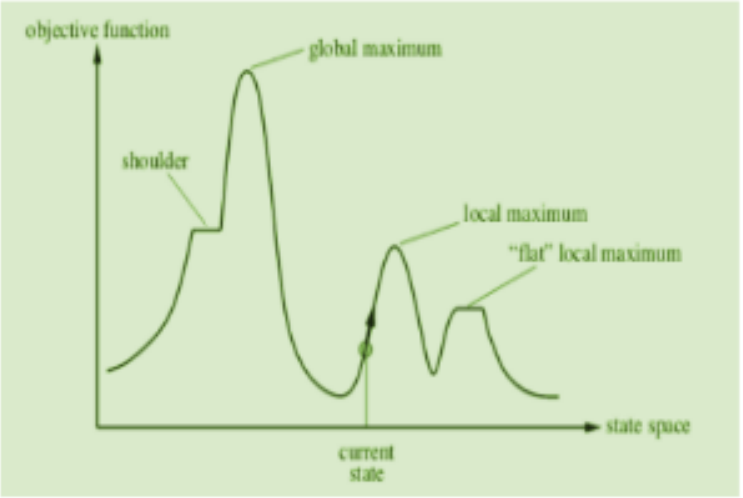
Algorithm:

1. Repeat k times:
   - Generate a random initial solution
   - Apply local search (e.g., Hill Climbing)
2. Return the best solution among all runs

Advantage: avoids local optima by restarting from new zones in the search space

Hill Climbing (Descent Method)





Hill Climbing (Descent Method)

Principle:

- Iteratively move to a better neighboring solution until no improvement is found.

Variants:

- **First Improvement:** Stop as soon as a better neighbor is found.
- **Best Improvement:** Explore all neighbors and choose the best one.

Drawback: May get trapped in local optima

GRASP (Greedy Randomized Adaptive Search Procedure)

Two phases:

1. **Construction phase:** Build a greedy randomized solution using a restricted candidate list (RCL)
2. **Local search phase:** Apply a local search to improve it

Process:

- Repeat construction + local search multiple times
- Keep the best solution found

Advantage: Combines randomness (exploration) with greedy selection (exploitation)

Exploration vs. Exploitation

Exploration

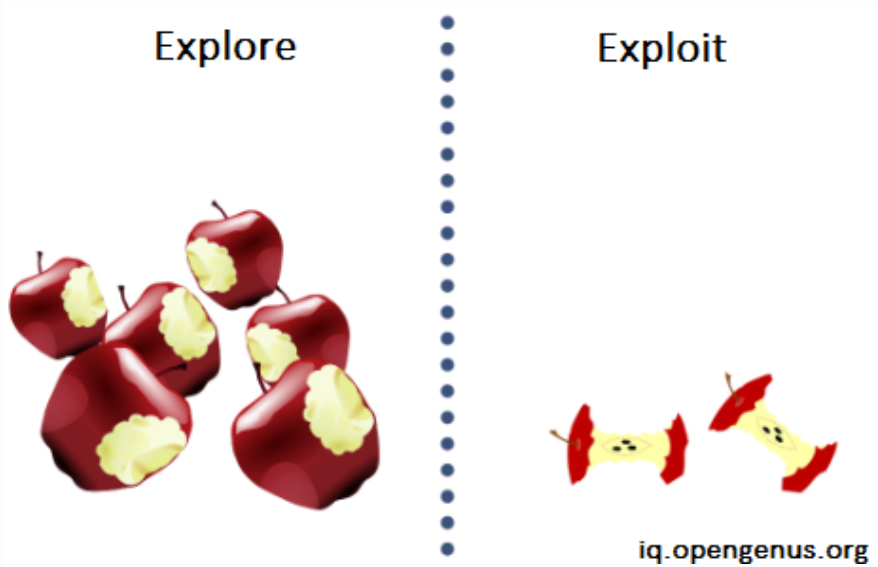
Searching new regions in the solution space.

Exploitation

Refining the current promising solution.

Trade-off:

- Multi-start: favors exploration
- Hill Climbing: favors exploitation
- GRASP: balances both


Explore
Exploit
iq.opengenus.org

Example Problem – Permutation Optimization

Objective: Minimize total flow × distance in a layout

Data:

- Permute 4 departments: A, B, C, D
- Flow matrix:

$$F = \begin{bmatrix} 0 & 3 & 0 & 2 \\ 3 & 0 & 2 & 1 \\ 0 & 2 & 0 & 4 \\ 2 & 1 & 4 & 0 \end{bmatrix}$$

Distance matrix for layout (positions 1–4):

$$D = \begin{bmatrix} 0 & 1 & 2 & 3 \\ 1 & 0 & 1 & 2 \\ 2 & 1 & 0 & 1 \\ 3 & 2 & 1 & 0 \end{bmatrix}$$

Goal: Assign A–D to positions 1–4 to minimize cost

Activity – Compare Two Local Search Variants

Initial solution: ABCD
Neighborhood: All permutations obtained by swapping 2 elements
Steps:

1. Apply **First Improvement**:
   - Scan neighbors in order and move to the first one that improves the cost
2. Apply **Best Improvement**:
   - Evaluate all neighbors and move to the one with the best improvement
3. Compare:
   - Number of evaluations
   - Final cost

Discuss: Which method converges faster? Which one gives a better solution?

Initial Solution Evaluation

Initial permutation: ABCD ⇒ A at position 1, B at 2, C at 3, D at 4
Flow matrix F and **Distance matrix** D are as previously defined.
Compute total cost:

$$\text{Cost} = \sum_{i,j} F_{ij} \cdot D_{\text{pos}(i),\text{pos}(j)}$$

$$= 3 \cdot 1 + 2 \cdot 1 + 1 \cdot 1 + 2 \cdot 1 + 4 \cdot 1 = 3 + 2 + 1 + 2 + 4 = \mathbf{12}$$

Generate Neighbors by Swapping

Neighborhood (swap 2 elements):

- BACD
- CBAD
- DBCA
- ACBD
- ADCB
- ABDC

We will now compute their costs.

Evaluate Neighbors (Manual Calculation)

Neighbor 1: BACD

- Positions: B-1, A-2, C-3, D-4
- Cost: $3 \cdot 1 + 3 \cdot 1 + 2 \cdot 1 + 1 \cdot 2 + 2 \cdot 1 + 4 \cdot 1 = \mathbf{14}$

Neighbor 2: CBAD

- C-1, B-2, A-3, D-4
- Cost: $2 \cdot 1 + 3 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 + 4 \cdot 1 = \mathbf{11}$

Neighbor 3: DBCA

- D-1, B-2, C-3, A-4
- Cost: $1 \cdot 1 + 3 \cdot 1 + 4 \cdot 1 + 2 \cdot 3 + 2 \cdot 1 = \mathbf{15}$

Continue Neighbor Evaluation

Neighbor 4: ACBD

- A-1, C-2, B-3, D-4
- Cost: $0 \cdot 1 + 2 \cdot 1 + 3 \cdot 2 + 4 \cdot 1 + 1 \cdot 1 = \mathbf{13}$

Neighbor 5: ADCB

- A-1, D-2, C-3, B-4
- Cost: $2 \cdot 1 + 1 \cdot 2 + 4 \cdot 1 + 3 \cdot 3 + 2 \cdot 1 = \mathbf{18}$

Neighbor 6: ABDC

- A-1, B-2, D-3, C-4
- Cost: $3 \cdot 1 + 1 \cdot 1 + 2 \cdot 1 + 4 \cdot 1 + 2 \cdot 1 = \mathbf{13}$

First vs Best Improvement

Initial cost: 12 (ABCD)

First Improvement:

- Try BACD → cost = 14 (worse)
- Try CBAD → cost = **11** (better) ⇒ accept and stop

Best Improvement:

- Evaluate all neighbors:
  - CBAD = **11** (best)
  - others: 13–18
- Choose CBAD

Result: Both methods select CBAD, but:

- First Improvement uses fewer evaluations
- Best Improvement gives a global view per iteration

Solution Encoding in Heuristics and Metaheuristics for Combinatorial Optimization

In metaheuristic algorithms, **solution encoding** is a crucial design step. It defines how candidate solutions are represented, which impacts feasibility, search efficiency, and the implementation of operators (mutation, crossover, neighborhood moves).

Knapsack problem

Problem: Select items to maximize total value without exceeding the knapsack's capacity.

Encoding

$$x = (x_1, x_2, \dots, x_n) \in \{0, 1\}^n$$

$$x_i = \begin{cases} 1 & \text{if item } i \text{ is selected} \\ 0 & \text{otherwise} \end{cases}$$

Feasibility Check Let w_i be the weight of item i, and C the capacity. A solution is feasible if:

$$\sum_{i=1}^{n} w_i x_i \leq C$$

Repair Strategy (if infeasible)

- Greedy removal: Remove items with lowest value-to-weight ratio until feasible.
- Penalization in fitness function.

Assignment Problem

Problem: Assign n agents to n tasks minimizing cost.

Encoding

$$\pi = (\pi_1, \pi_2, \dots, \pi_n)$$

$$\pi_i = j \Rightarrow \text{assign agent } i \text{ to task } j$$

Constraint: π must be a permutation of $\{1, 2, \dots, n\}$.

Feasibility Check

$$\text{Check that } |\{\pi_i\}| = n \text{ (i.e., all values are unique)}$$

Job Scheduling (e.g., Flow Shop)

Problem: Determine the processing order of n jobs on m machines to minimize makespan.

Encoding

$$\pi = (\pi_1, \pi_2, \ldots, \pi_n) \text{ — permutation of job indices}$$

Feasibility Check

- All jobs are scheduled once.
- No machine conflicts (checked during decoding via a scheduling model).

Traveling Salesman Problem

Problem: Find the shortest tour visiting each city exactly once and returning to the start.

Encoding

$$\pi = (\pi_1, \pi_2, \dots, \pi_n) \text{ — permutation of city indices}$$

Feasibility Check

$$\text{Verify } \pi \text{ is a valid permutation of } \{1, 2, \dots, n\}$$

Repair

- Remove duplicates, reinsert missing cities randomly or heuristically.

Transportation problem

Problem: Ship goods from supply points to demand points to minimize cost.

Encoding

$$X = [x_{ij}] \in \mathbb{R}^{m \times n}, \quad x_{ij} \geq 0$$

Feasibility Check

$$\sum_{j=1}^{n} x_{ij} \leq s_i \quad \text{(supply)} \quad \forall i$$

$$\sum_{i=1}^{m} x_{ij} \geq d_j \quad \text{(demand)} \quad \forall j$$

Repair

- Redistribute excess supply.
- Solve transportation sub-problems to restore feasibility.

Graph coloring

Problem: Color the vertices of a graph $G = (V, E)$ using minimum colors, such that no adjacent vertices share the same color.

Encoding

$$x = (x_1, x_2, \dots, x_n) \in \mathbb{N}^n$$

$$x_i = \text{color assigned to vertex } i$$

Feasibility Check

$$\forall (i, j) \in E, \quad x_i \neq x_j$$

Repair

- Reassign conflicting vertices to a different color.
- Add new colors if necessary.

As a conclusion:
Choosing the right encoding impacts:

- Search space size and structure.
- Feasibility enforcement.
- Operator design (e.g., crossover, mutation, neighborhood).

Feasibility can be ensured either by:

- Using a representation that always produces feasible solutions.
- Applying **repair heuristics**.
- Penalizing infeasible solutions in the fitness function.

Decoding solutions

Given an encoded solution $x \in \mathcal{X}$, decoding maps it to a full solution $s \in \mathcal{S}$ in the problem's domain:

$$\text{Decode} : \mathcal{X} \to \mathcal{S}$$

This step may involve:

- Constructing schedules or assignments.
- Computing start and end times of operations.
- Rebuilding feasible structures (e.g., tours, subsets, matchings).

Decoding Examples Across Problems

1. Knapsack Problem
   - **Encoding:** A binary vector $x = (x_1, x_2, \ldots, x_n)$ where $x_i \in \{0, 1\}$.
   - **Decoding:** Select items where $x_i = 1$, then compute total weight and value:

$$\text{Weight} = \sum_{i=1}^{n} x_i w_i, \quad \text{Value} = \sum_{i=1}^{n} x_i v_i$$

   - **Feasibility:** Check $\sum x_i w_i \leq C$ (capacity). If not, apply repair.

2. Travelling Salesman Problem (TSP)
   - **Encoding:** A permutation $\pi = (\pi_1, \pi_2, \dots, \pi_n)$.
   - **Decoding:** Build the tour visiting cities in the order π, return to start.
   - **Feasibility:** Any permutation is feasible; no decoding repair needed.

3. Assignment Problem
   - **Encoding:** A permutation π assigning task i to agent π_i.
   - **Decoding:** Construct the assignment matrix where $x_{i,\pi_i} = 1$.
   - **Feasibility:** Ensure each task and agent is used only once.

4. Scheduling Problem (e.g., Job Shop)
   - **Encoding:** A permutation of operations across machines.
   - **Decoding:** Use simulation or Gantt-chart decoding to determine start/end times, obeying:
     - Precedence constraints.
     - Machine availability.
     - Optional time windows or capacity limits.
   - **Feasibility:** Check for overlaps, precedence violations; use repair if needed.

5. Graph Coloring
   - **Encoding:** A vector $x = (c_1, \dots, c_n)$ where c_i is the color of vertex i.
   - **Decoding:** Assign each node its color.
   - **Feasibility:** Ensure adjacent nodes have different colors: $c_i \neq c_j$ if $(i, j) \in E$.

General Feasibility Checking After Decoding

After decoding, check:

- **Structural validity:** Is the solution complete and consistent?
- **Constraint satisfaction:** Are hard constraints respected?
- **Repairability:** If the decoded solution is infeasible, can it be repaired?

Why Decoding Matters

- It connects the abstract search space to the real problem space.
- It ensures correct evaluation of the objective and constraints.
- It allows metaheuristics to work with simpler, compact solution representations.

About scheduling

There are two common scheduling environments:

- **Flow Shop Scheduling:** All jobs follow the same machine order. Encoding is a **single permutation** of the n jobs:

$$\pi = (\pi_1, \pi_2, \dots, \pi_n), \quad \pi_i \in \{1, \dots, n\}$$

  This permutation defines the order in which jobs are processed on each machine.

- **Job Shop Scheduling:** Each job has its own sequence of machines. Encoding is an **operation-based permutation**, typically represented as:

$$\pi = (\pi_1, \pi_2, \dots, \pi_{n \cdot m})$$

where each job appears m times, respecting the order of its operations. This indicates the sequence of operations to schedule across all machines.

Feasibility Conditions

A solution is feasible if the following constraints are satisfied:

1. Precedence Constraint Operations of the same job must respect their processing order. If operation O_{ij} precedes $O_{i(j+1)}$, then:

$$\text{Start}(O_{i(j+1)}) \geq \text{End}(O_{ij}) = \text{Start}(O_{ij}) + p_{ij}$$

2. Machine Non-Overlap Constraint Each machine can process only one operation at a time. For any two operations O_a and O_b assigned to the same machine:

$$\text{End}(O_a) \leq \text{Start}(O_b) \quad \text{or} \quad \text{End}(O_b) \leq \text{Start}(O_a)$$

This ensures no overlapping intervals on any machine timeline.

3. Machine Capacity Constraint (Optional) If machines have capacity greater than one, we must ensure:

$$\sum_{\substack{O_{ij} \text{ active at } t \\ \text{on machine } k}} \text{load}_{ij} \leq \text{capacity}_k, \quad \forall t$$

Feasibility Checking Procedure

- Initialize machine availability timelines.
- Decode the operation sequence π:
  - For each operation, wait for both:
    - Completion of the preceding operation in its job.
    - Availability of its assigned machine.
  - Assign the operation to the earliest valid start time.

- After decoding, verify that:
  - No operations overlap on the same machine.
  - Job operation precedence is respected.
  - Time window or machine capacity constraints are satisfied.

Repair Strategies If the decoded solution is infeasible:

- Apply **shifting** to resolve overlaps.
- **Reorder** conflicting operations (e.g., using a local search or heuristic rule).
- Use **penalization** in the objective function to guide the search toward feasible regions.